CN 2025-11-18

Leggere l'Appendice A1.

Casi particolari di fattorizzazione LU

$A \in \mathbb{R}^{n \times n}$, $\det(A) \neq 0$, allora

$$A = LU \iff \forall k = 1, \dots, n \; ; \; \det(A_k) \neq 0.$$

- A è diagonale dominante;
- A simmetrica e definita positiva (sdp).

In particolare,

$$A \text{ sdp} \iff \exists\, L = (l_{ij}),\ D = \operatorname{diag}(d_1, \dots, d_n),$$
$$d_i > 0,\ i = 1, \dots, n : \quad A = LDL^T.$$

Algoritmo di fattorizzazione LDL^T per matrici sdp.

Se A è sdp, $A = A^T$ e quindi abbiamo necessità di specificare solo una sua porzione triangolare. Nel seguito, considereremo la parte triangolare inferiore di A. Ovvero, se $A = (a_{ij}) \in \mathbb{R}^{n \times n}$, allora considereremo solo gli elementi a_{ij}, con $i \geq j$ (quindi, della parte triangolare inferiore di A).

Per ottenere l'algoritmo di fattorizzazione, imponiamo che $\forall j = 1,\dots,n,\ i = j,\dots,n$, si abbia

$$a_{ij} = (LDL^T)_{ij}$$

Ricordiamo che, se $\underline{e}_i \in \mathbb{R}^n$ è l'i-esimo versore, allora:

$$a_{ij} = \underline{e}_i^T A \underline{e}_j = \underline{e}_i^T L D L^T \underline{e}_j$$

$$= (\underline{e}_i^T L) D (\underline{e}_j^T L)^T$$

$$= (l_{i1}\ l_{i2} \dots l_{ii}\ \overbrace{0 \dots 0}^{n-i}) \begin{bmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{bmatrix} \begin{bmatrix} l_{j1} \\ \vdots \\ l_{jj} \\ 0 \\ \vdots \\ 0 \end{bmatrix} \Big\} n-j$$

$$= (l_{i1}d_1\ \ l_{i2}d_2 \dots l_{ii}d_i\ 0 \dots 0) \begin{bmatrix} l_{j1} \\ l_{j2} \\ \vdots \\ l_{jj} \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$= \sum_{k=1}^{\min\{i,j\}=j} l_{ik} d_k l_{jk} = \sum_{k=1}^{j} l_{ik} l_{jk} d_k .$$

Abbiamo, quindi, concluso che:

$$a_{ij} = \sum_{k=1}^{j} l_{ik} l_{jk} d_k, \qquad j = 1,\dots,n,\ i \geq j.$$

$$= \sum_{k=1}^{j-1} l_{ik} l_{jk} d_k + l_{ij} \underbrace{l_{jj}}_{=1} d_j$$

$i=j$: $\quad a_{jj} = \sum_{k=1}^{j-1} l_{jk}^2 d_k + d_j$

$i=j+1, \dots, n$: $\quad a_{ij} = \sum_{k=1}^{j-1} l_{ik} l_{jk} d_k + l_{ij} d_j$

for $j = 1:n$

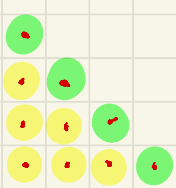
$$d_j = a_{jj} - \sum_{k=1}^{j-1} l_{jk} \underbrace{l_{jk} d_k}_{=v_k}$$

for $i = j+1 : n$

$$l_{ij} = \frac{a_{ij} - \sum_{k=1}^{j-1} l_{ik} \overbrace{l_{jk} d_k}^{v_k}}{d_j}$$

end

end



$n = 4$

Utilizzando un vettore di appoggio $\underline{r}$ di dimensione al più $n-1$, le operazioni richieste si verificano essere $\approx \frac{1}{3} n^3$.

Per quanto riguarda, invece, l'occupazione

di memoria, abbiamo che gli elementi della porzione triangolare inferiore di A possono essere sovrascritti con gli elementi significativi dei fattori L ed U:



Scriviamo una function Matlab che, dato in ingresso la matrice LU con l'informazione sui fattori L ed U di A, ed un termine noto b, calcola la soluzione del sistema lineare $A\underline{x} = \underline{b}$.



```
function   x = LUsolve(LU,b)
%
%  x = LUsolve(LU,b)
%
%   Risolve per i fattori L ed U della
%   matrice dei coefficienti del sistema lineare.
%
%  Input:
%     LU -  matrice con i fattori L ed U;
%      b -  termine noto;
%
%  Output:
%      x -  soluzione.
%                                   Rel. 2025-11-18.
%
if nargin<2, error('dati insufficienti'), end
[m,n] = size(LU);
if m~=n || n~=length(b)
      error('dimensione dati inconsistenti')
end
x = b(:);
for i=2:n                % fattore L
    x(i:n) = x(i:n) - LU(i:n,i-1)*x(i-1);
end
for i = n:-1:1           % fattore U
```

```
        if LU(i,i) == 0
            error('fattori U singolari')
        end

        x(i) = x(i) / LU(i,i);

        x(1:i-1) = x(1:i-1) - LU(1:i-1,i) * x(i);

    end

    return
```

Scrivere una function che, avendo in ingresso una matrice LD con l'informazione della fattorizzazione LDL^T di una matrice sdp A, nella porzione triangolare inferiore, risolva il sistema lineare $A\underline{x} = \underline{b}$, utilizzando tale fattorizzazione e il vettore $\underline{b}$ in ingresso.

```
function x = LDsolve(LD, b)
%
%   x = LDsolve(LD, b)
%
%   Risolve per i fattori L, D, L^t di
```

```
%   una matrice sdp.
%
%   Input:
%
%     LD  -  matrice con l'informazione dei
%            fattori L e D;
%      b  -  termine noto;
%
%   Output:
%      x  -  soluzione.
%                                   Rel. 2025-11-18.

if nargin<2, error('dati insufficienti'), end

[m,n] = size(LD);

if m~=n || n~= length(b)
        error('dati non compatibili')
end

x = b(:);

d = diag(LD);

if any(d<=0), error('D non positivo'), end

for i=2:n                    % fattore L
    x(i:n) = x(i:n) - LD(i:n,i-1)*x(i-1);   | axpy
end                                  ↑ accesso per colonne

x = x./d;                    % fattore D


for i=n-1:-1:1
```

```
        x(i) = x(i) - LD(i+1:n, i+1) * x(i+1:n);
                                  1
end
```

scal

accesso sempre per colonna alla matrice LD.

```
return
```